

Roadmap iDPSIR — Revisão 2

Versão de 29/09/2026 (branch roadmap-revisao2): modo estrutural adotado

Contents

Decisões já tomadas (não reabrir)	2
Decisões a confirmar	3
Fase 0 — Corrigir estado e segurança (antes de qualquer recurso novo)	3
Fase 1 — Modo estrutural: aresta = β , motor único e conversão (D17–D23)	5
Fase A — Modos de pressão e resposta e critério “até neutralizar” (D1 e D2 revisadas em 29/09)	7
Fase B — Relevância e prioridade dos Impactos (especificação v1.0)	10
Fase C — Camada de medida e gatilho do Estado (activation_threshold) (D7, D8, D11, D23)	12
Fase D — Crescimento (growth_rate) propagado pelas arestas de saída	15
Fase 2 — Consistência do método	17
Fase 3 — Robustez, desempenho e UX	18
Fase 4 — Publicação e manutenção	18
Ordem sugerida e dependências	18
Checklist final	19
Anexo — Justificativa do modo estrutural (29/09/2026)	19
X1 — Limiar, valor de referência e leitura estática $\times$ temporal (resolvido pelas D11, D20 e D23)	19
X2 — Modo estrutural: aresta = coeficiente de caminho padronizado β (adotado: D17–D23)	20
X3 — Acomodação do modo estrutural no roadmap (aplicada em 29/09)	22
X4 — Texto para o README e o tutorial: autorregulação (a aplicar na A7)	24
X5 — growth_rate $\times$ autorregulação (análise para a P3)	25

Objetivo: levar o iDPSIR da versão atual (main, commit 201d131) a uma versão que:

- (1) ofereça três tipos de resposta no tempo, escolhidos pelo usuário;
- (2) adote o **modo estrutural**: cada aresta é um coeficiente de caminho padronizado β , com faixa de incerteza, e o motor não tem mais o desconto c (Fase 1);
- (3) implemente a especificação “*Categorização dos nós de impacto & relevância por análise de loop*” v1.0 (decisões V1–V5 fechadas);
- (4) torne operacional o limiar dos Estados, em unidade real e disparado pelas Pressões; (5) propague o growth_rate de um nó para as arestas que saem dele; (6) corrija os bugs de estado e as inconsistências encontrados na avaliação de 25/09/2026.

Como usar com o Claude Code: leia antes o `CLAUDE.md` e o `README.md`. Cada item traz *motivação, o que fazer, arquivos, testes* e um critério **Pronto quando**, e cada item é uma unidade de trabalho (um commit ou um PR).

A ordem das fases importa: a Fase 0 corrige bugs nos mesmos módulos que as outras fases vão tocar, e a Fase 1 troca o motor sobre o qual as Fases A–D são escritas. Rode `testthat::test_dir("tests/testthat")` e **suba o app** a cada item que mexer em módulo Shiny, porque erros de Shiny só aparecem em runtime.

Protótipos numéricos: a pasta `prototipos_revisao2/` (na raiz do repositório) tem os scripts em Python que geraram os valores de referência citados aqui. Veja o `README.md` dela.

Modo estrutural (29/09/2026): adotado nesta revisão como **único** modo (decisões D17–D23). As fases abaixo já foram reescritas para acomodá-lo. A justificativa está no **Anexo, seções X2 e X3**, e as conferências em R em `prototipos_revisao2/modo_estrutural.R`.

Conferência em R (29/09/2026): os valores de referência foram conferidos contra o motor R real de main (201d131) com `prototipos_revisao2/conferencia_em_R.R`. Todos bateram. Nesta conferência foram feitas quatro correções: a fórmula da tendência exógena (D2), a referência de código da 0.4, uma ressalva sobre o exemplo do PDF (B3) e o impacto da Fase C no exemplo Gnanapragasam (C6). Depois, a D1 e a D2 foram revisadas e a D16 foi cancelada: “até neutralizar” virou critério de parada da simulação (A2), com novos valores de referência na A6.

Decisões já tomadas (não reabrir)

#	Decisão	Origem
D1	A pressão e a resposta têm, cada uma, dois modos escolhidos pelo usuário: permanent ou impulse. “Até neutralizar” não é um modo da resposta : é um critério de parada da simulação (duração “N janelas fixas” ou “até a resposta neutralizar o Impacto”, com máximo de janelas).	Usuário, 25/09; revisado 29/09
D2	Padrões na tela: pressão permanent, resposta permanent (hoje é impulse) e duração “ até neutralizar ”, com máximo de 50 janelas , editável.	Usuário, 25/09; revisado 29/09
D3	(Cancelada em 29/09 pela D19.) Antes: o fator λ da leitura estática ficava como estava. Agora não há desconto c: a atenuação de uma resposta distante vem do produto dos $\beta < 1$ ao longo do caminho.	Usuário, 25/09; cancelada 29/09
D4	Na estrutura DPSIR do app, ciclos só existem via Resposta (arestas $I \rightarrow R$). Uma cadeia causal sem Resposta tem no máximo $D \rightarrow P \rightarrow S \rightarrow I$.	Usuário + <code>schema_allowed_connections()</code>
D5	(Substituída em 29/09 pelas D17 e D18.) A magnitude de β fica, como regra, em $(0, 1]$; $\beta > 1$ gera aviso, não bloqueio.	Usuário, 25/09; substituída 29/09
D6	Especificação de relevância v1.0 com V1–V5 fechadas: <code>endpoint_class</code> , valor v (swing weights), D max-normalizado, p multiplicativo, blend de centralidade só como fallback. A eficácia foi alterada pela D15.	PDF v1.0
D7	O que dispara o gatilho do Estado = soma com sinal de $\beta \times$ nível das Pressões e Respostas que chegam ao Estado, em DP (nível de D/P recortado em ≥ 0). No modo estrutural, na leitura estática, isso é o próprio desvio do Estado.	Usuário (Q5), 25/09; ajustada 29/09
D8	Gatilho binário : liga ou desliga todas as arestas de saída do Estado.	Usuário (Q6), 25/09
D9	O crescimento (<code>growth_rate</code>) fica no nó , como hoje. A influência das arestas de saída cresce na proporção $(1+g)$ por meio do valor do nó. O peso da aresta não é multiplicado (evita dupla contagem).	Usuário (Q9), 25/09
D10	Um nó com <code>growth_rate > 0</code> cresce como tendência exógena mesmo fora do cenário de pressão, nas duas rodadas (baseline e cenário).	Usuário (Q10), 25/09
D11	Gatilho na simulação temporal: os dois critérios (“carga na janela” e “desvio acumulado do Estado”) ficam disponíveis na tela. O usuário escolhe qual usar e pode ver os dois resultados lado a lado. Padrão: desvio acumulado (revisado em 29/09, P1): o limiar representa uma perda ou acúmulo total, como no estoque de peixe ou no contaminante no sedimento.	Usuário (Q7), 25/09; padrão revisado 29/09
D12	Reach: mostrar os dois, o topológico (como hoje) e o efetivo no cenário, que não atravessa Estados com gatilho fechado, lado a lado.	Usuário (Q8), 25/09
D13	(Revisada em 29/09 pelas D19 e D22.) Os exemplos passam ao modo estrutural com a classe de cada aresta revisada com o usuário. O app e os exemplos publicados no GitHub e na demo shinylive são atualizados juntos. Arquivo antigo é convertido na importação ($\beta = \lambda \cdot w$, preservando os números antigos), com aviso, em vez de bloqueado.	Usuário (Q1), 25/09; revisada 29/09

#	Decisão	Origem
D14	Importância dinâmica D = sempre pela propagação do app (<code>propagate()</code>), a mesma da leitura de suficiência); Levins não entra como alternativa. No modo estrutural, isso é o equilíbrio de Levins em forma padronizada ($\beta_{ij} = a_{ij}/ a_{ii} $; conferido em R), então o aviso de divergência sai.	Usuário (Q2), 25/09; revisada 29/09
D15	Eficácia da resposta = cobertura (a força do slider). A incerteza da aresta (faixa de β , D21) entra só na incerteza (reamostragem de p e da suficiência), não multiplica a eficácia. Isto altera a V5 do PDF v1.0, que deve ser atualizado.	Usuário (Q3), 25/09
D16	(Cancelada em 29/09.) Antes: modo <code>until_neutralized</code> com “manter”/“retirar” a medida. Com a D1 revisada, a resposta não liga e desliga; é a simulação que para quando o impacto é neutralizado.	Usuário (Q4), 25/09; cancelada 29/09
D17	Aresta = coeficiente de caminho padronizado β : quantos desvios-padrão o destino muda quando a origem muda 1 DP. O sinal vem de <code>interaction_type</code> , a magnitude fica em <code>weight</code> , e o segundo campo é a faixa de incerteza (<code>weight_low/weight_high</code>). O r^2 é só atalho ($ \beta = \sqrt{r^2}$) e diagnóstico. O slope bruto é só conversor, na camada dos nós.	Usuário, 29/09
D18	Sem dados, o usuário escolhe uma classe: fraca < 0,3 (valor 0,15, faixa 0–0,3), moderada 0,3–0,6 (0,45, padrão), forte ≥ 0,6 (0,80, faixa 0,6–1,0). O sinal nunca tem padrão. Arestas com classe ou padrão ficam marcadas na tabela e no relatório.	Usuário, 29/09
D19	Só o modo estrutural . Sai o desconto c (slider “How far to trace the effect”) e a tabela “Does it hold up across how far the effect is traced”. O motor é único sobre B , com efeito $(I - B)^{-1}p - p$, e exige $p(B) < 1$. Arquivos antigos são convertidos na importação.	Usuário, 29/09
D20	Persistência temporal $a = 1 - \text{self_regulation} + g$, como hoje (sobre g , ver P3). <code>self_regulation</code> aceita [0, 1] e tem padrão 0,5 para nós novos (meia-vida de 1 janela, acúmulo de $2 \times$ no equilíbrio; revisado em 29/09). Arquivos antigos mantêm o valor gravado. $sr = 0$: o nó acumula como estoque; $sr = 1$: sem memória. A leitura estática, $(I - B)^{-1}p - p$, coincide com o equilíbrio temporal só nos nós com $sr = 1$. Com $sr < 1$, o equilíbrio temporal é $(sr \cdot I - B)^{-1}p$ (Levins com diagonal $-sr$); com $sr = 0$, não há equilíbrio (conferido em R).	Usuário, 29/09
D21	O confidence das arestas vira a faixa de β . Sem faixa informada, vale a faixa da classe. Confiança antiga κ é convertida pela regra atual de reamostragem: $\beta \cdot [1 - 0,5(1 - \kappa), 1 + 0,5(1 - \kappa)]$.	Usuário, 29/09
D22	Os exemplos (Fisheries, Mangi, Gnanapragasam e <code>sample</code>) ficam só no modo estrutural , com a classe de cada aresta revisada com o usuário.	Usuário, 29/09
D23	Camada de medida : nós podem ter valor de referência (> 0) e variação típica (DP ou CV). O limiar do Estado é um nível na unidade do nó, sem teto em 0–1 , convertido para $z = (\text{limiar} - \text{ref})/\text{DP}$, com direção pelo sinal de z . Pressão e resposta podem passar de 100% ou ser informadas em unidade real.	Usuário, 29/09

Decisões a confirmar

As perguntas Q1–Q10 de 25/09 viraram as decisões D7–D16. Em 29/09, D1 e D2 foram revisadas, D16 foi cancelada, e as decisões do modo estrutural (D17–D23) revisaram D3, D5, D7, D13, D14 e D15.

P1 e P2 foram respondidas pelo usuário em 29/09. A P3 está em aberto:

#	Pergunta	Decisão	Onde entra
P1	Critério padrão do gatilho na temporal	Desvio acumulado (revisa o padrão da D11)	C3
P2	Arquivo antigo que, convertido, fica com $ \beta > 1$	Aviso e revisão , sem bloqueio	1.5
P3	Em aberto (29/09) . <code>growth_rate</code> é redundante com a autorregulação no motor atual. Separar os dois? g sai da diagonal e vira só tendência exógena do nível de referência (Anexo X5)	Proposta: separar	Fase D, A

Fase 0 — Corrigir estado e segurança (antes de qualquer recurso novo)

Bugs encontrados por leitura de código. Todos têm correção pequena.

0.1 — Grafo não é invalidado após editar nós/arestas

Motivação: a aba Graph desenha `nodes()/edges()` ao vivo. Metrics, Scenarios e Report usam `graph()`, congelado no último “Build”. Se o usuário remove um nó depois do Build, as métricas e os cenários continuam contando esse nó. **O que fazer:** em `mod_data.R`, `observeEvent(list(rv$nodes, rv$edges, rv$schema), { rv$graph <- NULL; rv$graph_message <- "Network changed - rebuild in step 5" }, ignoreInit = TRUE)`. Hoje `rv$graph` só é zerado nas ações do Start (linhas ~236–349). O botão Next do passo 5 já bloqueia quando `graph()` é NULL. **Arquivos:** `R/modules/mod_data.R`. **Pronto quando:** construir, remover um nó e tentar abrir o Explore exige reconstruir.

0.2 — Estado dos módulos não é reiniciado ao carregar outro projeto

Motivação: `current_scenario`, `saved_scenarios`, `comparison_selection`, `graph_snapshots` e `scenario_counter` sobrevivem a `start_new`, `start_savepoint` e `start_merge`. O relatório do projeto B passa a mostrar cenários e imagens do projeto A. **O que fazer:** `mod_data` expõe um `reactiveVal` “época” que é incrementado a cada carga. `mod_responses`, `mod_graph` e `mod_report` observam a época e zeram o próprio estado. **Arquivos:** `mod_data.R`, `mod_wizard.R`, `mod_responses.R` (l. ~269, 625), `mod_graph.R` (l. ~464), `mod_report.R`. **Pronto quando:** salvar cenário e snapshot no projeto A e carregar o B deixa o Report de B vazio.

0.3 — Vetores de cenário alinhados por nome

Motivação: `p_D/p_R` viram posicionais (`as.numeric` descarta nomes em `temporal.R`:254–255 e `report.R`:309). Um cenário salvo antes de adicionar um nó recicla vetores e gera números errados sem nenhum erro. **O que fazer:** alinhar sempre por `p[rownames(W)]` (`NA` → 0) e dar `stop()` se algum nome não bater. Invalidar `saved_scenarios` ao reconstruir o grafo (junto com 0.1). **Testes:** caso com vetor fora de ordem e caso com nó a mais em `test-temporal.R` e `test-sufficiency.R`.

0.4 — Re-salvar savepoint não pode apagar o cenário

Motivação: `current_scenario_state` (`R/modules/mod_responses.R`:750, consumido pelo download do savepoint em `mod_wizard.R`) lê `input$active_*`, que só existe depois que a aba Scenarios foi aberta. Se o usuário carrega um savepoint com cenário, corrige um rótulo e salva, o cenário some. **O que fazer:** quando os inputs forem NULL, usar o estado restaurado (`restored()`). **Pronto quando:** carregar, editar um rótulo e salvar sem abrir Scenarios preserva o `scenario_state`. **Nota:** hoje `current_scenario_state` também não grava os modos temporais nem o número de janelas. A A5 precisa acrescentá-los aqui.

0.5 — Erros não podem derrubar a sessão

O que fazer: `tryCatch + showNotification(type = "error")` em “Apply scenario” (`mod_responses.R`:271–335), na simulação temporal e no `downloadHandler` do relatório (`mod_report.R`:269–304).

0.6 — Savepoint JSON passa pela mesma validação do CSV

Motivação: `read_savepoint()` (`io.R`:179–185) só chama `normalize_*`. Um `confidence: 3` gera `runif(min > max)` → NaN. Um peso negativo com `interaction_type: negative` inverte o sinal. **O que fazer:** rodar `preflight_import()` sobre `raw$nodes/raw$edges`, com as mesmas mensagens bloqueantes. Depende de 1.1 para os novos campos de aresta (β , faixa, classe).

0.7 — Sinal ausente é erro, não “+1”

O que fazer: em `validate.R`:219–223, uma célula `interaction_type` vazia vira erro bloqueante. Na normalização, peso e confiança NA recebem os defaults documentados (hoje o default só vale quando a coluna inteira falta).

0.8 — Escapar HTML nos tooltips

Motivação: `build_node_tooltip()/build_edge_tooltip()` (`graph.R:220–247`) interpolam `label`, `descriptor` e `reference` crus. Um `savepoint` compartilhado com `<img onerror=...>` executa script, inclusive na demo pública. **O que fazer:** `htmltools::htmlEscape()` em cada campo antes do `glue`. **Testes:** novo `test-graph.R` com rótulo contendo `<script>`.

Fase 1 — Modo estrutural: aresta = β , motor único e conversão (D17–D23)

Situação atual (verificada):

- O peso só precisa ser > 0 , sem limite superior (`validate.R:231–247`).
- O modal de aresta usa `numericInput("Weight (> 0)", min = 0.01, step = 0.5)` (`mod_data.R:731`), com padrão 1.
- A leitura estática aplica um desconto global $\lambda = c$ (sem ciclo) ou $c/p(W)$ (com ciclo), controlado pelo slider “How far to trace the effect” (`effect_horizon`).
- Todos os exemplos usam pesos acima de 1: Fisheries 0,5–3; Mangi 1–3; Gnanapragasam 1–2,5; `sample_edges.csv` 1–3.

Justificativa e conferências: Anexo X2 e X3; script `prototipos_revisao2/modo_estrutural.R`.

1.1 — Campos da aresta e validação (D17, D18, D21)

O que fazer:

- `weight` passa a ser a magnitude de β . O sinal continua em `interaction_type`.
- Novas colunas opcionais: `weight_low`, `weight_high` (faixa de incerteza) e `strength_class` (`weak | moderate | strong`).
- **Validação:**
 - bloquear `weight <= 0` ou NA;
 - **aviso** (não bloqueio) para `weight > 1`: um β padronizado pode passar de 1 em regressão múltipla com origens correlacionadas;
 - bloquear faixa inválida (`low > weight` ou `high < weight`).
- **Padrões por classe (D18):**
 - `weak` = 0,15 (faixa 0–0,3);
 - `moderate` = 0,45 (0,3–0,6), **padrão**;
 - `strong` = 0,80 (0,6–1,0).
- Sem faixa informada, vale a faixa da classe; sem classe, vale `moderate`.
- `confidence` sai do formulário (D21). Em arquivos antigos, é convertido para faixa pela regra atual de reamostragem: $\beta \cdot [1 - 0,5(1 - \kappa), 1 + 0,5(1 - \kappa)]$.
- Validação por nó: aviso quando $\sum \beta^2$ das arestas que chegam > 1 (“a rede explicaria mais de 100% da variação deste fator; revise os β ”).

Arquivos: `validate.R` (`preflight` e `normalize`), `io.R`, `schema.R` (campos conhecidos), `graph.R` (`tooltip` e espessura por $|\beta|$).

Testes: `test-validate.R` (faixas, classes, aviso > 1 , soma de β^2) e `test-io.R` (`round-trip`).

1.2 — Formulário de aresta (D17, D18)

O que fazer: em `mod_data.R:731` (modal de aresta):

- seletor “Strength”: Weak / Moderate (default) / Strong / “I have a value”;
- com “I have a value”: entrada de β , ou os atalhos:
 - $r^2 \rightarrow |\beta| = \sqrt{r^2}$ (aviso: vale para origens independentes);
 - r^2 e $n \rightarrow$ faixa pelo erro-padrão $\sqrt{((1 - r^2)/(n - 2))}$;
- a faixa aparece preenchida pela classe e pode ser editada;
- na tabela de arestas e no relatório, marcar as arestas cujo valor veio de classe ou padrão.
- Validar também em `mod_data.R:781`.

1.3 — Motor único sobre B (D19)

O que fazer:

- `effect_matrix(g)` devolve B, a matriz de β com sinal, `B[to, from]` e diagonal zero. Substitui `build_signed_matrix()`.
- `propagate(B, p) = (I - B)-1p - p`, **sem** `c` e **sem** λ .
- **Validação bloqueante:** $\rho(B) \geq 1$ (“the network’s feedback loops amplify without bound; reduce the β values on the loop”), com a lista das arestas do ciclo dominante.
- **Remover o alcance c:**
 - o slider `effect_horizon`;
 - `sufficiency_reach_over_c()`, `format_reach_over_c_table()` e a tabela “Does it hold up across how far the effect is traced” na tela e no relatório;
 - o argumento `c` de `sufficiency()`, `sufficiency_confidence()` e dos demais.
- `effect_horizon` em savepoint antigo é lido só para a conversão (1.5) e depois descartado.
- `strength_to_neutralize` continua uma razão linear (sem gatilho), agora sem `c`.
- Comentário de cabeçalho de `sufficiency.R` reescrito: o efeito total é a regra de rastreamento de caminhos da path analysis. A atenuação de uma resposta distante vem do produto dos $\beta < 1$, o que cumpre o papel da antiga D3.

Testes (test-sufficiency.R):

- cadeia $P \rightarrow S \rightarrow I$ com $\beta -0,7/-0,6$: efeito 0,42;
- exemplo do X2.4: $P1 \rightarrow I = 0,42$, $R \rightarrow I = -0,336$, mitigação de 80%, força para neutralizar 125%;
- bloqueio de $\rho(B) \geq 1$;
- os testes com `c` são removidos ou reescritos.

1.4 — Incerteza pela faixa de β (D21)

O que fazer: `sufficiency_confidence()` e, depois, `prediction_reliability()` (B4) reamostram cada β uniformemente em `[weight_low, weight_high]`, e não mais por `confidence × spread`. Usar `withr::with_seed()` (ver 2.4). **Testes:** faixa degenerada (`low = high = \beta`) dá 100% ou 0%; reprodutibilidade com semente.

1.5 — Arquivos antigos: conversão na importação (substitui a D13)

O que fazer: em `read_savepoint()`, `import_matrizes()` e `merge_savepoints()`, arquivo sem `metadata$weight_mode = "structural"`:

- converter cada peso para $\beta = \lambda \cdot w$, com o λ da leitura antiga (`c/p(W)` com ciclo, `c` sem ciclo; `c = effect_horizon` do savepoint ou 0,5). **Isso preserva exatamente os números da leitura estática antiga**, porque a antiga já era $(I - \lambda W)^{-1}p - p$;

- converter confidence para faixa (regra da 1.1) e marcar todas as arestas como “convertidas”;
- mostrar um aviso com o λ usado e as arestas com $|\beta| > 1$ (redes sem ciclo com peso > 2). Não bloqueia (P2): as arestas ficam marcadas “revisar” na tabela de arestas e no relatório até o usuário confirmar ou editar o valor;
- gravar `metadata$weight_mode = "structural"` e `metadata$converted_from = list(lambda, c)`;
- no merge, converter cada arquivo antes de juntar.

Testes (test-io.R): o savepoint Mangi antigo convertido reproduz os números atuais de `sufficiency()` com $c = 0,5$ (tolerância $1e-9$); um CSV sem ciclo com peso 3 gera aviso de $|\beta| = 1,5$.

1.6 — Exemplos no modo estrutural (D22, substitui a antiga 1.3)

O que fazer:

- Revisar **com o usuário** a classe de cada aresta (fraca/moderada/forte), partindo da conversão da 1.5. Arquivos: `data/*_edges.csv`, `data/sample_edges.csv` e `docs/*.idpsir.json` (Fisheries, Mangi, Gnanapragasam).
- Validar $\rho(B) < 1$ e $\Sigma \beta^2 \leq 1$ por nó.
- Rodar a comparação da B3 nos três exemplos.
- **Publicar no GitHub:** os exemplos novos substituem os antigos em main, e a demo shinylive é republicada. Conferir o link “Try it live” e os downloads do tutorial.
- Refazer os números do tutorial e as figuras `docs/example_gnanapragasam_*.png`, junto com a A7.
- Atualizar os fixtures de `tests/testthat` que usam peso > 1 ou c .

Pronto quando:

- toda aresta tem β e faixa (informados, por classe ou convertidos);
- o motor não tem mais c ;
- $\rho(B) \geq 1$ é bloqueado;
- arquivos antigos convertem preservando os números;
- os exemplos carregam sem aviso;
- os testes passam e o tutorial bate número a número com o app.

Fase A — Modos de pressão e resposta e critério “até neutralizar” (D1 e D2 revisadas em 29/09)

Contexto (verificado por simulação da cadeia $P \rightarrow E \rightarrow I$ com $R \rightarrow P$, pesos 1):

- O motor é $x(t+1) = (1 - sr + g) \cdot x(t) + B \cdot x(t) + p$ (D20), com B da Fase 1. Os valores da A6 usam $\beta = 1$, sem o estabilizador λ , e continuam válidos. Cada nó funciona como um estoque que acumula o que chega a cada janela.
- `impulse` não age só uma janela: vira um **nível constante** do nó, que continua agindo a cada janela enquanto o nó não tiver `self_regulation`.
- `permanent` soma +1 a cada janela, então o esforço cresce sem limite e pode ultrapassar a meta.
- Nenhuma opção atual para a simulação quando o Impacto zero.
- O Impacto só converge para zero, sem ultrapassar, quando P , E e I têm `self_regulation`.

A1 — Seletores de modo com ajuda

O que fazer: em `mod_responses.R:438–442`, os dois seletores (pressão e resposta) oferecem as mesmas duas opções, com `permanent` selecionado nos dois:

Valor interno	Rótulo	Texto de ajuda
permanent	Added every window (default)	"The push is added again every window, so its effect keeps building up. Use for an ongoing pressure or a management effort that keeps being applied. A response can overshoot and push the Impact below zero."
impulse	Applied once and held	"The push is applied in window 1 only; the level it creates stays in the system (it fades only if that factor has self-regulation). Use for a one-off event or measure."

Mostrar a ajuda do modo selecionado logo abaixo de cada seletor (helpText reativo).

A2 — Duração da simulação: "N janelas fixas" ou "até neutralizar"

O que fazer: em `simulate_temporal_pair()` (`temporal.R:157`):

- `mode_R = c("permanent", "impulse")`, com `permanent` como primeiro valor e, portanto, o padrão do `match.arg`. `mode_D` fica como está.
- Novos argumentos `stop_rule = c("until_neutralized", "fixed")` e `max_windows = 50`. Com "fixed", `windows` é usado como hoje.
- **Critério de parada** (`until_neutralized`):
 - **Impactos considerados:** os alcançados pelas respostas ativas (`response_reach()` com os `active_ids`) e que pioram no baseline naquela janela (`x_baseline[I](t) > tol_abs`). Isso evita parar antes de o problema chegar: nas primeiras janelas o Impacto ainda é 0 por causa do atraso da cadeia.
 - A simulação para na **primeira janela t** em que existe pelo menos um Impacto considerado e **todos** os considerados têm `x_scenario[I](t) <= tol_abs`, ou seja, chegaram a zero ou cruzaram para o lado bom.
 - **Não usar a tolerância relativa da A3 para parar.** Contraexemplo verificado em R: pressão permanent, resposta impulse, sem autorregulação. O Impacto sobe sem parar (6, 10, 15, 21...), mas o baseline sobe mais rápido, e na janela 60 o Impacto fica abaixo de 5% do baseline. A tolerância relativa diria "neutralizado".
 - Em janelas discretas o Impacto costuma pular o zero (ex.: $5 \rightarrow 0 \rightarrow -14$). O critério " $\leq tol_abs$ " cobre o pulo.
 - Se não parar até `max_windows`, o resultado traz `neutralized_at = NA` e a tela mostra "Not neutralized within N windows".
 - Se nenhum Impacto for alcançado pela resposta, avisar e rodar `max_windows`.
- O resultado ganha `neutralized_at` (janela ou NA) e `stop_rule`. Com "fixed", `neutralized_at` também é calculado (a primeira janela que cumpre o critério dentro das N), para informar na tela.
- Atualizar os defaults em `report.R:313` e `mod_responses.R:652` (`% | % "permanent"`).

UI:

- Seletor "Simulation length", com as opções "Until the response neutralizes the Impact (default)" e "Fixed number of windows".
- `numericInput` "Maximum windows" (padrão 50, faixa 1–200) ou "Windows", conforme a opção.
- No gráfico temporal (`scenario_plots.R`), marcar a janela de neutralização com uma linha vertical. Na tabela, destacar a linha dessa janela.

A3 — Tolerância no veredito temporal

Motivação: hoje "Neutralized" exige $|I| \leq 10^{-9}$. Um Impacto que converge para 0,03 (contra baseline 37) aparece como "Partial" para sempre. **O que fazer:** `format_temporal_table(..., tol_rel = 0.05, tol_abs = 1e-9)`. O Impacto está neutralizado quando $|net| \leq \max(tol_abs, tol_rel * |baseline|)$. Adicionar um `numericInput` "Neutralization tolerance (% of baseline)" com padrão 5, e informar o valor no relatório. A leitura estática continua exata (é álgebra, não dinâmica). **Cuidado:** esta tolerância só rotula a tabela; ela não decide a parada (ver A2). Quando o baseline cresce mais rápido que o cenário, a razão cai abaixo de 5% mesmo com o Impacto piorando. Por isso a tabela só mostra "Neutralized (relative)" se $|net|$ não estiver crescendo em relação à janela anterior; caso contrário, mostra "Partial". Cobrir o contraexemplo da A2 com teste.

A4 — Baseline sem nenhuma resposta (opcional)

Motivação: arestas $I \rightarrow R$ também agem no baseline, então o baseline não é “sem resposta”: o próprio Impacto aciona R nas duas rodadas. **O que fazer:** checkbox “Baseline without any response (ignore Impact $\rightarrow$ Response links)”. Padrão desmarcado, para compatibilidade. Quando marcado, a rodada baseline usa W com as linhas dos nós Response zeradas.

A5 — Persistir modos, duração e tolerância

O que fazer: gravar `temporal_mode_pressure`, `temporal_mode_response`, `temporal_stop_rule`, `temporal_max_windows`, `temporal_windows`, `temporal_tol_rel` e `baseline_without_response` no `scenario_state` do savepoint (`io.R build_savepoint/read_savepoint`; o estado atual da tela vem de `current_scenario_state`, `mod_responses.R:750`) e nos cenários salvos.

- Um savepoint antigo sem esses campos recebe os novos defaults, **exceto** `temporal_stop_rule`: quando o savepoint já traz `temporal_windows`, ele vira “fixed”, para reproduzir o resultado salvo.
- Um valor antigo “impulse” é preservado.

A6 — Testes de referência (cadeia $P \rightarrow E \rightarrow I$, $R \rightarrow P$, pesos 1)

Valores do motor R real (`confere_em_R.R`, 29/09/2026). O teste deve reproduzi-los com tolerância $1e-6$.

Janelas fixas, pressão permanent:

Caso	Janela	Baseline I	Cenário I
resposta impulse, sem self_regulation	5 / 10 / 30	10 / 120 / 4060	6 / 36 / 406
resposta permanent, sem self_regulation	5 / 8 / 10	10 / 56 / 120	5 / -14 / -90
resposta impulse, self_regulation 0,3 em P, E, I	10 / 30	22,8599 / 36,9588	2,9648 / 0,0267

“Até neutralizar” (`tol_abs = 1e-9`, `max_windows = 50`): janela em que a simulação para e valor de I nela.

Pressão	Resposta	Sem self_regulation	self_regulation 0,3 em P, E, I
permanent	permanent	janela 7 ($I = 0$)	janela 6 ($I = -0,67$)
permanent	impulse	não neutraliza (NA)	não neutraliza (NA); $I \approx 0,03$ na janela 30
impulse	impulse	janela 6 ($I = 0$)	janela 5 ($I = -0,16$)
impulse	permanent	janela 6 ($I = -5$)	janela 5 ($I = -1,16$)

- Com tolerância de 5%, o caso resposta impulse com self_regulation 0,3 aparece como “**Neutralized (relative)**” na tabela na janela 30 (A3), mas a simulação “até neutralizar” não para, porque I continua positivo e não chega a zero. Documentar essa diferença na ajuda.
- Contraexemplo da A3: pressão permanent, resposta impulse, sem self_regulation. Na janela 60, $|net| < 5\%$ do baseline, mas o Impacto está crescendo; a tabela **não** pode mostrar “Neutralized”.
- Coerência com a leitura estática (D20):** cadeia $P \rightarrow S \rightarrow I$ com $\beta -0,7/-0,6$, $sr = 1$ em todos os nós e pressão permanent converge para $I = 0,42$, igual a $(I - B)^{-1}p$. Com $sr = 0,5$, converge para $3,36 = (0,5 \cdot I - B)^{-1}p$. Os dois casos entram como teste.
- self_regulation passa a aceitar 1 (hoje a faixa é $[0, 1)$).

A8 — Autorregulação: padrão, guia e aviso (D20)

O que fazer:

- Padrão 0,5** para nós novos (`validate.R normalize`, `mod_data.R create_empty_nodes_table()` e formulário). Arquivos antigos e CSVs mantêm o valor gravado. Célula em branco num CSV novo recebe 0,5, com aviso no preflight.

- Faixa aceita: $[0, 1]$ (hoje $[0, 1]$).
- **Guia no formulário de nó:** junto ao campo, a tabela do Anexo X4 (autorregulação $\times$ meia-vida $\times$ acúmulo no equilíbrio $\times$ exemplo) e uma frase: “0 = accumulates like a stock; 1 = no memory”.
- **Aviso na validação:** quando todos os nós têm autorregulação 0 (“nothing in this network recovers on its own; every effect accumulates window after window”), e quando um Estado com limiar tem autorregulação 0 (o desvio acumulado nunca se desfaz sozinho: o gatilho, uma vez aberto, só fecha se uma resposta empurrar o Estado de volta).
- **Testes:** padrão 0,5 em nó novo; arquivo antigo preserva 0; os dois avisos.

A7 — Documentação

O que fazer: atualizar a seção temporal do README e do tutorial com os dois modos e o critério “até neutralizar”, usando a tabela acima como exemplo didático. **Incluir no README e no tutorial o texto sobre autorregulação do Anexo X4** (pedido do usuário, 29/09). Mensagem central: “*quem faz o Impacto convergir é a autorregulação da cadeia; o modo da resposta decide o esforço; o critério de parada diz em que janela o problema foi resolvido*”. **A mudança de padrão (D2) altera o gráfico temporal do exemplo Gnanapragasam:** regenerar a figura e os números.

Pronto quando: os dois seletores e a duração aparecem com ajuda; a autorregulação tem padrão 0,5, guia e avisos (A8); permanent, permanent e “até neutralizar” (50 janelas) são os padrões em UI, motor e relatório; os testes A6 passam; o savepoint preserva as escolhas; e o tutorial reflete o novo padrão.

Fase B — Relevância e prioridade dos Impactos (especificação v1.0)

Avaliação da especificação frente ao código atual:

- O exemplo numérico da seção 4 **confere**. Autovalores $-0,7, -0,7, -0,6, -0,5$. $\delta = -A^{-1}u = [+2,00; -2,67; -3,43; -2,95]$ para P, S, I1, I2.
- Três pontos exigem adaptação:
 1. **D por Levins ($-A^{-1}$) exige A estável.** Isso só acontece se todo nó tiver `self_regulation > 0`, mas o padrão do app é 0. Decisão D14: usar só a propagação. No modo estrutural (Fase 1), a propagação é o próprio equilíbrio de Levins em forma padronizada, então a divergência desaparece (ver B3).
 2. **V5 (atenuar a aresta de saída da pressão-alvo)** usa outro modelo de resposta. No app, a resposta é um nó com arestas $R \rightarrow \text{alvo}$, empurrado por uma força. Decisão D15: a força da resposta = cobertura (slider), e a confiança fica só na incerteza (B5).
 3. **Vocabulário de sinal:** o PDF usa *increases/triggers/improves/reduces/mitigates*, e o app usa *positive/negative*. Aceitar os dois (B6).
- A Fase 1 (β e motor único) é pré-requisito.

B1 — Campos novos no nó de Impacto

O que fazer:

- `endpoint_class`, categórico com valores `ecological | service | welfare` e padrão `ecological`.
- `value_v`, numérico em $[0, 1]$ com padrão 1. Só é editável quando `endpoint_class` em `{service, welfare}`; para `ecological` é forçado a 1 (V1).
- Só se aplicam a nós de Impacto. Em outras categorias ficam em branco e a validação bloqueia valores preenchidos. **Arquivos:** `schema.R` (colunas opcionais de nó), `validate.R` (vocabulário e faixa), `io.R` (`normalize`, `savepoint` e `CSV`), `mod_data.R` (modal de nó: campos condicionais à categoria), tabela de nós, `report.R`. **Testes:** `test-validate.R` e `test-io.R` com round-trip dos dois campos.

B2 — Ajuda para elicitar v (swing weights, V1)

O que fazer: no passo Nodes, um botão “Elicit values (swing weights)”. Ele abre um modal que lista os Impactos service/welfare. O usuário ordena pela importância de “levar do pior ao melhor nível” e dá 100 ao primeiro e 0–100 aos demais; o app normaliza por $\div 100$ e grava em value_v. Editar value_v à mão continua possível.

B3 — Importância dinâmica D (decisão D14)

O que fazer: novo arquivo R/relevance.R com `dynamic_importance(g, p_D)`.

- $\delta_I = \text{propagate}(\text{effect_matrix}(g), p_D)[I]$, a mesma leitura da suficiência. Com a Fase C, usar a matriz com os gatilhos aplicados.
- $D_I = |\delta_I| / \max_J |\delta_J|$ sobre os Impactos (V2). Se todos forem 0, $D = 0$ e aparece um aviso.
- Sem blend de centralidade. Sem aviso de divergência com Levins (ver abaixo).
- Nota de método na tela e no relatório: “D é o efeito total da pressão sobre cada Impacto (path analysis), equivalente ao equilíbrio de Levins da especificação v1.0 quando os coeficientes são padronizados.”

Validação (conferida em R, prototipos_revisao2/modo_estrutural.R):

Leitura	δ_{I1} (ecológico)	δ_{I2} (bem-estar)	D I1	D I2	Líder
Levins (PDF v1.0, $-A^{-1}u$)	-3,43	-2,95	1,00	0,86	I1
Estrutural, $\beta_{ij} = a_{ij}/ a_{ii} $ e push $\div a_{PP} $	-3,43	-2,95	1,00	0,86	I1
Estrutural com a_{ij} cru (sem dividir pela autorregulação)	-0,72	-0,82	0,88	1,00	I2
Antigo propagate, $c = 0,5$	-0,18	-0,33	0,55	1,00	I2

- **Conclusão:** a inversão registrada antes não era Levins $\times$ propagação, e sim coeficiente cru $\times$ padronizado. Um β de regressão já é padronizado, e com ele a leitura do app é a da especificação v1.0.
- **Para o PDF v1.0:** explicar a equivalência na seção 4 e dizer que os a_{ij} da especificação correspondem a $\beta_{ij} \cdot |a_{ii}|$.
- **Ressalva:** a rede do exemplo tem uma aresta direta $P \rightarrow I2$, que o schema padrão do app não permite. O teste monta a matriz diretamente.
- Rodar D nos três exemplos no modo estrutural (Fase 1.6) e registrar no tutorial.
- **Teste de referência:** exemplo da seção 4 com β padronizado: $D(I1) = 1,00$ e $D(I2) = 0,86$ (tolerância $1e-2$).

B4 — Confiabilidade p (sign determinacy empírica)

O que fazer: `prediction_reliability(g, p_D, n_sim = 300, seed = 42)`. Reamostrar cada β uniformemente na sua faixa (D21, 1.4), como `sufficiency_confidence()`, e medir a fração de simulações em que $\text{sign}(\delta_I)$ é igual ao do caso base.

- Pular reamostragens com $p(B) \geq 1$ e informar quantas foram puladas.
- Usar `withr::with_seed()`, e não `set.seed()` global (ver 2.4). **Testes:** $p = 1$ quando todas as faixas são degeneradas (low = high = β).

B5 — Eficácia da resposta e gap (decisão D15)

O que fazer:

- Eficácia da resposta $R = \text{cobertura}$ = o slider de força (0–100% $\rightarrow$ 0–1). A confiança da aresta **não** multiplica a eficácia.
- A incerteza entra só como faixa de β (D21): é ela que define a reamostragem em p (B4) e em `sufficiency_confidence()`.
- $\text{gap}_I = \min(1, |\text{net}_I| / |\text{worsening}_I|)$, com net e worsening de `sufficiency()` (com os gatilhos da Fase C quando houver limiar). É o mesmo que $|\delta \text{ com resposta}| / |\delta \text{ sem resposta}|$.
- Se $|\text{worsening}_I| \leq \text{tol}$, o gap é NA e o Impacto aparece como “Not affected” (ver 2.1).
- Documentar na tela e no relatório que esta implementação difere da V5 do PDF v1.0 (eficácia = cobertura $\times$ confiança), e atualizar o PDF.

B6 — Aliases do vocabulário de sinal

O que fazer: na importação (CSV/JSON), mapear `increases|triggers|improves|sustains` → `positive` e `reduces|mitigates|decreases` → `negative`. Informar no preflight quantas linhas foram mapeadas. O modal continua oferecendo só `positive/negative`.

B7 — Relevância, prioridade e tela

O que fazer:

- $\text{relevance_I} = v_I \cdot D_I \cdot p_I$, estável.
- $\text{priority_I} = \text{relevance_I} \cdot \text{gap_I}$, móvel.
- Nova seção “**Impact prioritization**” na aba `Scenarios`, logo abaixo da tabela de suficiência. Ela depende do cenário de pressão (D) e do de resposta (gap).
 - Tabela com Impacto, `endpoint_class`, D, p, v, relevância, gap e prioridade, ordenada por prioridade.
 - Gráfico de barras horizontais da relevância com uma marca na prioridade, como na figura 7.4 do PDF, reaproveitando o estilo de `scenario_plots.R`.
 - Nota de método sobre D (B3).
- Seção correspondente no relatório (`report.R`), depois de “Response sufficiency”.
- Download CSV da tabela.

B8 — Exemplo novo trazido pelo usuário (fora do escopo agora)

O exemplo portuário (tipo Santos) da seção 7 do PDF **não** será implementado nesta revisão. O usuário vai carregar outro exemplo. Quando ele chegar, validar com as Fases 1 e B (β por classe, `endpoint_class/value_v`) e só então adicioná-lo aos exemplos e ao tutorial.

Pronto quando: os campos B1 fazem round-trip no savepoint, o exemplo da seção 4 passa como teste, e a seção de priorização aparece na tela e no relatório com a nota de método sobre D.

Fase C — Camada de medida e gatilho do Estado (`activation_threshold`) (D7, D8, D11, D23)

Pedido: o limiar do Estado decide se a cadeia continua. Se o desvio do Estado causado pelas Pressões passar do limiar, as arestas Estado→Impacto ligam; caso contrário, a cadeia para no Estado. O desvio pode vir da **soma** de duas ou mais Pressões. O limiar é um nível real do Estado (ex.: perda de 40% do estoque; um contaminante no limite legal, 400% acima do fundo), sem teto em 0–1.

Situação atual (verificada no código):

- **Leitura estática (`sufficiency.R`):** ignora o limiar. `build_signed_matrix()` nunca chama `build_threshold_matrix()`.
- **Simulação temporal (`temporal.R:120, apply_threshold_gate()`):** compara o nível acumulado $|x_S(t)|$ / `reference_value` com um limiar em `[0, 1]`, sem direção.
- `build_threshold_matrix()` (`loop_analysis.R:150`) já aplica o limiar do nó a **todas** as arestas de saída do Estado. Essa parte serve como está.
- Validação: limiar em `[0, 1]` e só em nós State (`validate.R:144–172`).

C0 — Camada de medida dos nós (D23)

O que fazer:

- Nós ganham sd opcional: a variação típica, na mesma unidade de reference_value. O formulário aceita DP ou CV (%) e converte.
- reference_value passa a exigir > 0 (hoje só ≠ 0) e a distinguir “em branco” de “1”. Guardar NA; o motor usa 1.
- activation_threshold vira um **nível na unidade do nó**, sem teto em 0–1. Exige reference_value e sd no Estado. O limiar em DP é $z = (\text{limiar} - \text{ref}) / \text{DP}$.
- Direção pelo sinal de z**: $z < 0$ abre quando $x_S \leq z$ (ex.: estoque caindo); $z > 0$ abre quando $x_S \geq z$ (ex.: contaminante subindo).
- Limiar antigo** (fração f de 0–1, sem direção): converter para o modo “ambas as direções”, $|x_S| \geq f \cdot \text{ref} / \text{DP}$. Usar DP = ref quando ausente, o que reproduz o critério atual $|x| / \text{ref} \geq f$.
- Pressão e resposta aceitam força acima de 100% (slider até 1000% ou entrada numérica), ou em unidade real quando o nó tem ref e DP. O push em DP é o valor informado dividido pelo DP.
- Tabela “Change relative to initial values”** (temporal e relatório): nível = $\text{ref} + \text{DP} \cdot x$ por nó com ref informado, % de mudança sem e com resposta, e janela em que o limiar foi cruzado. A opção “mostrar todos” inclui os nós sem ref.
- Aviso de extrapolação**: quando o cenário leva algum nó além de ± 3 DP, lembrar que β é linear e que a relação real pode saturar ou ter forma dose-resposta.

C1 — Desvio do Estado e estado do gatilho (novo R/triggers.R, D7 e D8)

O que fazer:

- state_deviation(g, p): x_S no cenário, em DP, com sinal.
 - Estático**: $x_S = \sum_j \beta_{Sj} \cdot \ell_j$, em que ℓ_j é o nível total da origem j (push + efeito total), com D/P recortados em ≥ 0 (D7).
 - No modo estrutural, a carga imediata e o desvio estático do Estado são o mesmo número.
- state_gates(g, p): data.frame com id, z, direction, deviation e open. Sem limiar = sempre aberto.
- gate_matrix(B, gates): zera as colunas dos Estados com gatilho fechado.
- Com ciclo $I \rightarrow R \rightarrow P$, o desvio depende do próprio gatilho. Iterar até estabilizar (no máximo 20 vezes). Se oscilar, marcar unstable = TRUE e usar gatilho aberto (conservador), com aviso.

Referência (modo estrutural, conferida em R). Rede $P1 \rightarrow S$ e $P2 \rightarrow S$ ($\beta -0,3$ cada), $S \rightarrow I$ ($\beta -0,8$), $R \rightarrow P1$ ($\beta -0,5$ ou $-0,8$). Limiar de S em $z = -0,4$ (S cai):

Caso	Desvio de S sem resposta	Com resposta	Piora em I	Líquido em I
Só P1 a 100%	-0,30 (fechado)	—	0	0
P1 + P2 a 100%	-0,60 (aberto)	—	+0,48	+0,48
P1 + P2, $R \rightarrow P1$ β 0,5	-0,60 (aberto)	-0,45 (aberto)	+0,48	+0,36
P1 + P2, $R \rightarrow P1$ β 0,8	-0,60 (aberto)	-0,36 (fechado)	+0,48	0
P1 + P2 a 70%	-0,42 (aberto)	—	+0,336	+0,336
P1 + P2 a 60%	-0,36 (fechado)	—	0	0

- Consequência central**: uma resposta pode neutralizar um Impacto **fechando o gatilho**, mesmo quando a mitigação linear seria só parcial (linha “ β 0,8”).
- Isso quebra a linearidade**: com gatilho, o líquido tem que ser calculado com os dois cenários juntos.
- Os valores diferem do protótipo gatilho.py (0,120 / 0,105 / 0,084) porque o protótipo aplicava o desconto $c = 0,5$, que sai na D19.

C2 — Leitura estática com gatilho (sufficiency.R)

O que fazer:

- `worsening = propagate(gate(B, p_D), p_D)`.
- `net = propagate(gate(B, p_D + p_R), p_D + p_R)`.
- `mitigation = net - worsening`.
- **Só quando algum Estado tem limiar.** Sem limiar, o caminho continua exatamente igual ao da Fase 1 (teste de regressão byte a byte).
- `strength_to_neutralize`: com gatilho, buscar por bisseção a menor força da resposta (0–1000%) que zera o líquido, e informar se a neutralização vem “por mitigação” ou “por fechamento do gatilho”.
- Novo valor de veredito: **“Neutralized (below threshold)”**.
- `sufficiency_confidence()`: reavaliar os gatilhos em cada reamostragem e reportar também a % de simulações em que cada gatilho ficou aberto.

C3 — Simulação temporal com os dois critérios de gatilho (D11, D20)

Com a persistência da D20, o Estado acumula ($sr < 1$). Por isso “o que chega nesta janela” e “o desvio acumulado” são diferentes, e os dois critérios continuam fazendo sentido. Coincidem só quando o Estado tem $sr = 1$.

O que fazer: em `apply_threshold_gate()`, acrescentar `gate_mode = c("load", "state_level")`, os dois em DP e com direção (C0).

- `state_level`: $x_S(t)$, o desvio acumulado, comparado com z . **É o padrão (D11, P1)**.
- `load`: $\sum_j \beta_{Sj} \cdot x_j(t)$ (o que chega em t) comparado com z . É uma opção.
- Nos dois modos, o gatilho é reavaliado a cada janela (reversível, sem trava).
- **Na tela:** seletor “Trigger criterion” com “Accumulated State level (default)”, “Load arriving at the State” e “Compare both”.
 - Em “Compare both”, rodar a simulação duas vezes e mostrar lado a lado: duas colunas na tabela por Impacto e janela, e dois painéis no gráfico.
 - Destacar as janelas em que os dois critérios discordam.
- Mostrar, por janela, se cada gatilho estava aberto: coluna na tabela e marca no gráfico.
- Gravar o critério no `scenario_state` e nos cenários salvos; o relatório mostra o critério usado.
- Texto de ajuda: “Load = o que chega nesta janela passou do limite? Accumulated = o Estado já se desviou além do ponto crítico somando todas as janelas? Uma pressão fraca e constante pode nunca disparar por carga, mas disparar por acumulação.”
- **Teste de referência (q7.py, β 0,3/0,8, limiar em $z = -0,4$):** P1 sozinha a 100%, impulso. Por carga nunca abre (Impacto 0); por nível acumulado abre na janela 4. Reconferir em R com a convenção de direção da C0.

C4 — Interface e relatório

O que fazer:

- Tabela “State triggers” em Results (Scenarios): Estado, limiar (unidade real e z), desvio só da pressão, desvio com resposta, antes (aberto/fechado), depois, e “a resposta fecha o gatilho?”.
- No grafo, Estados com limiar ganham uma marca (borda dupla ou ícone). Na aba Scenarios, as arestas de saída de um Estado fechado aparecem tracejadas em cinza.
- Formulário de nó: “Activation threshold (in the factor’s own units): the State only passes its effect on to Impacts once it crosses this level. Needs the reference value and typical variation.”
- Relatório: seção “State triggers”, depois de “Response sufficiency”.
- Tutorial e README: explicar o gatilho com o exemplo da tabela acima e com o exemplo de contaminação.

C5 — Validação e Reach

O que fazer:

- **Aviso na validação:** limiar sem `reference_value` ou `sd` no Estado.
- **Aviso:** $|z|$ maior que o maior desvio possível com todas as Pressões a 100% (“este gatilho nunca dispara com pressões de até 100%”).
- **Aviso:** Estado com limiar e nenhuma aresta de Pressão chegando.
- **Reach (D12):** mostrar lado a lado “N fatores alcançáveis na rede” (topológico) e “M alcançados neste cenário” (sem atravessar Estados com gatilho fechado). Listar os Estados fechados que explicam a diferença, também na comparação de cenários e no relatório.

C6 — Testes de referência

- Os seis casos da tabela da C1, com tolerância $1e-6$.
- Conversão de limiar antigo: $f = 0,15$ com $ref = 100$ e sem DP reproduz o critério atual $|x|/ref \geq 0,15$.
- Direção: um contaminante com $z = +8$ não abre com desvio negativo.
- Regressão: rede sem limiar dá resultado idêntico ao da Fase 1 em `sufficiency()` e `simulate_temporal_pair()`.
- **Exemplo Gnanapragasam** (`activation_threshold = 0,15` em S1, $ref = 100$): o critério padrão (desvio acumulado) é o mesmo de hoje, e o limiar antigo convertido na C0 (ambas as direções, $DP = ref$) reproduz a regra atual. Por isso, os números mudam só pela Fase 1 (β) e pela revisão de classes (1.6). Registrar antes e depois e refazer o tutorial.
- Ciclo $I \rightarrow R \rightarrow P$ com gatilho oscilante: `unstable = TRUE` e aviso.
- `strength_to_neutralize` por bisseção no caso de fechamento do gatilho.

Pronto quando:

- o limiar de um Estado, em unidade real e com direção, decide na leitura estática e na temporal se a cadeia passa para os Impactos;
- a soma de Pressões conta;
- pressões acima de 100% funcionam;
- a tela e o relatório mostram desvio, limiar e estado de cada gatilho, além da tabela em unidades reais;
- os testes C6 passam e redes sem limiar não mudam nada.

Fase D — Crescimento (`growth_rate`) propagado pelas arestas de saída

Atenção (P3, em aberto): no motor atual, `growth_rate` é redundante com a autorregulação: só a diferença $g - sr$ entra no passo (conferido com o motor real). A proposta do Anexo X5 separa os dois: g sai da diagonal e vira só a tendência exógena do nível de referência. Se aceita, D1, D2 e D5 são ajustados como descrito no X5, antes de implementar esta fase.

Pedido: se um nó tem `growth_rate` positivo, as arestas que saem dele devem aumentar de intensidade na mesma proporção a cada janela.

Situação atual (verificada no código e simulada, `crescimento.py`):

- O motor temporal faz $x(t+1) = x + g \cdot x + B \cdot x + p$ (`temporal.R:139`, com B da Fase 1). O crescimento fica no **valor do nó**, e a aresta transmite $w \cdot x$.
- Por isso a influência que sai do nó **já cresce na proporção $(1+g)$** a cada janela, quando o nó tem um desvio. Exemplo: D com $g = 0,10$ e aresta $D \rightarrow P$ de peso 0,5, impulso em D. A contribuição em P é $0,50 \rightarrow 0,55 \rightarrow 0,605 \rightarrow \dots$, razão 1,10 por janela.
- **Multiplicar também o peso da aresta por $(1+g)^t$ contaria o crescimento duas vezes:** a razão viraria 1,21 por janela (decisão D9: o crescimento fica no nó).
- **Lacuna real 1:** o crescimento só age sobre um desvio que já existe. Um Driver com `growth_rate` que não está no cenário de pressão fica em zero para sempre. O tutorial do Gnanapragasam contorna isso ativando D3 a 30% à mão.

- **Lacuna real 2:** a leitura estática ignora o `growth_rate` por completo. Isso está documentado e é coerente com uma leitura de um único instante.
- **Lacuna real 3:** o usuário não vê em lugar nenhum a intensidade efetiva das arestas crescendo.

Contribuição que chega em P por janela (D → P com $w = 0,5$; $g = 0,10$):

Modelo	Janela 1	2	3	4	8	Razão por janela
Atual (g no nó), impulso em D	0,50	0,55	0,61	0,67	0,97	1,10
g só no peso da aresta, impulso em D	0,50	0,55	0,61	0,67	0,97	1,10
g no nó e na aresta (dupla contagem)	0,50	0,61	0,73	0,89	1,90	1,21
Tendência exógena, D fora do cenário (proposta D2)	0,05	0,11	0,17	0,23	0,57	→ 1,10
Atual, D fora do cenário	0	0	0	0	0	—

D1 — Representação do crescimento (decisão D9: fica no nó)

O que fazer:

- Manter g no nó.
- Documentar no código e no tutorial que a intensidade efetiva de cada aresta de saída é $w \cdot x_{origem}(t)$, que cresce na proporção $(1+g)$ por janela.
- Acrescentar um teste que garanta que a razão é 1,10, e não 1,21.

D2 — Tendência exógena sem precisar de semente (decisão D10)

O que fazer:

- Em `simulate_temporal_pair()`, todo nó com $g > 0$ recebe, nas **duas** rodadas, uma entrada exógena que faz o seu nível seguir $reference_value \cdot ((1+g)^t - 1)$. Em DP (C0), a entrada é $ref \cdot g / DP = g / CV$ por janela; sem DP, usa $DP = ref$ (entrada g , como no D5). Como o motor já aplica o termo $g \cdot x$ ao nó, isso equivale a uma entrada **constante** de $reference_value \cdot g$ por janela: $x(t+1) = (1+g) \cdot x(t) + ref \cdot g$ dá exatamente $ref \cdot ((1+g)^t - 1)$. **Não** usar $ref \cdot g \cdot (1+g)^{(t-1)}$: somada ao termo $g \cdot x$, ela contaria o crescimento duas vezes (na janela 2 daria $2g(1+g)$ em vez de $g(2+g)$). Correção feita na conferência em R de 29/09/2026.
- Se o nó também estiver no cenário de pressão, o push se soma a essa tendência.
- Como a tendência entra nas duas rodadas, ela não envia a comparação cenário $\times$ baseline. Ela só faz o mundo de fundo se mover.
- Checkbox “Apply growth trends even to factors outside the pressure scenario”, padrão ligado. Gravar no `scenario_state`.
- Atualizar o exemplo Gnanapragasam: a ativação manual de D3 a 30% deixa de ser necessária. Refazer os números do tutorial.

D3 — Mostrar a intensidade efetiva

O que fazer:

- Na simulação temporal, adicionar a tabela “Edge intensity by window”, com a aresta (origem → destino) e $w \cdot x_{origem}(t)$ por janela.
- Mostrar só as arestas que saem de nós com $g \neq 0$ ou que estão num gatilho da Fase C.
- Ter a opção de download em CSV.
- No gráfico temporal, uma legenda curta: “Factors with a growth trend: D3 (+3%/window)”.

D4 — Interações com outras fases

O que fazer:

- **Fase C (gatilho):** com crescimento, a carga num Estado sobe a cada janela, então o gatilho pode abrir numa janela posterior. Mostrar a janela em que cada gatilho abriu pela primeira vez.
- **Fase A (“até neutralizar”):** com uma tendência crescente, a neutralização pode ser temporária: o Impacto volta a subir depois da janela em que a simulação parou. Na tela, oferecer “Continue N more windows” e documentar.
- **Leitura estática:** continua sem crescimento (instante único). Opcionalmente, um parâmetro “Evaluate at window T” que multiplica o push de cada nó com $g \neq 0$ por $(1+g)^T$. É baixa prioridade.
- **Validação:** hoje `growth_rate` não tem limite. Bloquear valores ≤ -1 (o nó inverteria de sinal) e avisar acima de 0,5 por janela.

D5 — Testes de referência

- D→P com $w = 0,5$, $g = 0,10$, impulso em D: contribuição em P de 0,50 / 0,55 / 0,605 nas janelas 1–3, razão 1,10 exata e nunca 1,21.
- D fora do cenário com tendência ligada: 0,05 / 0,105 / 0,1655 nas janelas 1–3. Com a tendência desligada: 0 em todas.
- Baseline e cenário recebem a mesma tendência: com resposta nula, o líquido é igual ao baseline.
- Regressão: com `growth_rate = 0` em todos os nós, o resultado é idêntico ao atual.

Pronto quando: o crescimento de um nó aumenta a influência das suas arestas de saída na proporção $(1+g)$ por janela, sem dupla contagem, mesmo quando o nó não está no cenário. A intensidade efetiva aparece na tela, os testes D5 passam e redes sem crescimento não mudam nada.

Fase 2 — Consistência do método

#	Item	Onde	O que fazer
2.1	“Neutralized” quando a pressão nem alcança o Impacto	<code>sufficiency.R:176</code> , <code>temporal.R:361</code>	Veredito “Not affected” quando $ worsening \leq tol$. Não entra na confiança nem na prioridade. A janela 0 mostra “—”.
2.2	Confiança exibida não é a do veredito exibido	<code>mod_responses.R:73</code>	A matriz testa cada resposta sozinha a 100%. Acrescentar uma linha “Planned scenario” que reamostra <code>p_R</code> real com as forças dos sliders.
2.3	Caminhos passando por mais de uma Resposta	<code>pathways.R:39–51</code>	<code>all_simple_paths(..., cutoff = nível_destino - nível_origem)</code> pelo order do schema, ou excluir nós Response do meio do caminho. Assim a busca respeita D-P-S-I-R (D4). Aplicar <code>max_paths</code> durante a enumeração.
2.4	Reprodutibilidade	<code>sufficiency.R:211</code> , <code>metrics.R:96</code>	Trocar <code>set.seed()</code> global por <code>withr::with_seed()</code> . Fixar semente no Louvain. Adicionar <code>withr</code> a <code>required_packages</code> .
2.5	Estabilizador temporal encolhe a autorregulação	<code>temporal.R:213</code> , 248	Remover o estabilizador λ (<code>stability_cap</code>): no modo estrutural, os β são dados e não podem ser reescalados, e $p(B) < 1$ já é exigido (1.3). Manter só o aviso <code>temporal_stability_note()</code> quando $p(a \cdot I + B) > 1$. Recalcular os testes afetados.
2.6	Categorias com nome fixo	<code>sufficiency.R</code> , <code>temporal.R</code> , <code>metrics.R</code> , <code>reach.R</code> , <code>relevance.R</code>	Derivar os papéis (Impact, Pressure, Response) da coluna role do schema, para que renomear no passo Model não esvazie as tabelas.
2.7	Score de caminho	<code>pathways.R:67–69</code>	Efeito do caminho = produto dos β com sinal , que é o efeito indireto exato daquele caminho na path analysis. Mostrar o efeito, o sinal e a faixa (produto dos limites da faixa).

Fase 3 — Robustez, desempenho e UX

#	Item	Onde	O que fazer
3.1	Wizard frágil entre máquinas	mod_wizard.R:18–44, mod_data.R	Trocar os conditionalPanel guiados pelo numericInput escondido por tabsetPanel(type = "hidden") + updateTabsetPanel().
3.2	Grafo redesenha inteiro a cada arrasto ou slider	mod_graph.R:269, 391	isolate(positions()), debounce() nos sliders, visNetworkProxy para mudanças cosméticas.
3.3	Simulação temporal recalcula a cada passo do slider	mod_responses.R:484	bindEvent() num botão "Run simulation".
3.4	Matriz de confiança lenta no shinylive	sufficiency.R	withProgress e número de simulações configurável (100/300/1000). Sem a varredura de c (D19), o custo já cai.
3.5	Sliders de cenário voltam ao padrão após qualquer edição	mod_responses.R:188	isolate() dos valores atuais ao re-renderizar.
3.6	Upload antigo reimportado após erro	mod_data.R:140–243	Mensagens num uiOutput separado do card do fileInput.
3.7	Erros em azul	mod_data.R:193, 842	alert-danger para erros.
3.8	Snapshot falha em silêncio	mod_report.R:87, 182	Enviar o erro via Shiny.setInputValue e notificar.
3.9	Remoção sem confirmação	mod_data.R:588, mod_responses.R:655	Modal "N arestas serão removidas" e aviso ao sobrescrever um cenário com o mesmo nome.
3.10	Modais aceitam valores inválidos	mod_data.R:644–657	Validar NA e faixas (uncertainty, controllability, self_regulation em [0, 1], value_v, β , faixa de β , ref > 0, DP > 0), auto-laço e aresta duplicada.
3.11	Duas paletas conflitantes	mod_graph.R:273, schema.R:17	Uma só fonte de paleta; Okabe-Ito como padrão.
3.12	IDs de nó usados como IDs de input	mod_responses.R	Usar índice ou make.names() para os inputs.
3.13	Cenários e snapshots salvos fora do savepoint	io.R	Persistir saved_scenarios no .idpsir.json.

Fase 4 — Publicação e manutenção

- **6.1 do roadmap anterior:** LICENSE (MIT), CITATION.cff, DESCRIPTION com todas as dependências de global.R, incluindo withr e htmltools.
- **6.2 renv:** renv.lock só para desenvolvimento e CI. O auto-install do global.R continua como fallback para runGitHub().
- **CI:** job de testthat antes do export shinylive; fixar a versão de R e do shinylive.
- **Código morto:** mover para legacy/ (ou uma taggit) o seguinte: sufficiency_reach_over_c(), format_reach_over_c_table() e build_signed_matrix() (substituída por effect_matrix()); as partes não usadas de loop_analysis.R, **exceto** build_interaction_matrix, self_regulation_diagonal, build_threshold_matrix e build_press_vector; draw_trajectory_plot, draw_sensitivity_plot e o storyboard antigo de scenario_plots.R; responses.R exceto get_feedback_categories; R/dpsir/; mod_communities.R; mod_data_ui. Corrigir o README (find_response_targets não é usada).
- **Testes que faltam:** pathways.R, report.R (incluindo escape), import_matrizes, rede vazia, NA em sinal/peso, savepoint fora de faixa e desalinhamento p–W.
- **CLAUDE.md:** mover o histórico de bugs (~270 KB) para docs/CHANGELOG.md. Manter no CLAUDE.md só o resumo operacional e as decisões D1–D23 deste roadmap.

Ordem sugerida e dependências

1. **Fase 0** (0.1–0.8): cerca de 1–2 dias. O 0.6 depende de 1.1 para os novos campos de aresta, então pode ser feito junto com 1.1.

2. **Fase 1** (modo estrutural): 3–4 dias. 1.1–1.5 antes de qualquer outra fase, porque trocam o motor. A 1.6 (exemplos, com revisão de classes junto com o usuário) pode andar em paralelo com A–D, mas precisa terminar antes de refazer o tutorial.
3. **Fase A** (modos de resposta e “até neutralizar”): 1–2 dias. A A7 regenera as figuras do tutorial junto com a 1.6.
4. **Fase B** (relevância): 3–4 dias. Mais simples que antes: sem aviso de Levins, reamostragem pela faixa de β .
5. **Fase C** (camada de medida e gatilho): 3–4 dias. A C0 (ref, DP, limiar em unidade real, força > 100%) vem antes de C1–C6.
6. **Fase D** (crescimento): 1–2 dias, junto com ou depois da Fase A (mesmo motor temporal) e depois da C0 (tendência em DP).
7. **Fases 2, 3 e 4**, em paralelo conforme a disponibilidade. A 2.5 (remover o estabilizador λ) entra junto com a Fase A.

Checklist final

- ☐ `testthat::test_dir("tests/testthat")` passa.
- ☐ App sobe localmente e na demo shinylive.
- ☐ Os três exemplos, no modo estrutural (β por classe revisada), carregam sem aviso no app local e na demo shinylive.
- ☐ Um savepoint antigo convertido reproduz os números da leitura estática antiga.
- ☐ PDF de relevância atualizado (seção 4 e 7.4: equivalência entre Levins e β padronizado; V5 com eficácia = cobertura).
- ☐ Tutorial e README batem número a número com o app (β e classes, modos de pressão e resposta, critério “até neutralizar”, priorização, gatilho em unidade real, crescimento).
- ☐ Decisões D1–D23 registradas no CLAUDE.md.
- ☐ Relatório informa o modo da resposta, a tolerância, o critério do gatilho, as arestas com valor por classe ou convertido, e as sementes usadas.

Anexo — Justificativa do modo estrutural (29/09/2026)

Este anexo registra a discussão que levou ao modo estrutural. **As decisões resultantes já estão nas fases acima (D17–D23).** As perguntas P1 e P2, que ficaram abertas, foram respondidas em 29/09 (ver “Decisões a confirmar”).

X1 — Limiar, valor de referência e leitura estática × temporal (resolvido pelas D11, D20 e D23)

Leitura proposta pelo usuário:

- `reference_value` é o valor inicial, positivo, de qualquer nó.
- `activation_threshold` é uma fração desse valor: é o ponto de virada do Estado.
- Exemplo: um estoque de peixe com `ref = 10` e limiar 0,4 tem o gatilho aberto quando o estoque perde 4 unidades.
- O limiar vale tanto na leitura estática quanto na temporal.

Conflito com D7/D11:

- A D7 compara com o limiar a *carga* de uma janela (peso × intensidade), sem usar o ref. Isso só é coerente quando `ref = 1`.
- No exemplo Gnanapragasam (S1 com `ref = 100` e limiar 0,15), cada critério dá uma resposta diferente:
 - carga sem ref: abre já na janela 1;
 - carga ÷ ref: nunca abre;
 - nível acumulado ÷ ref: abre quando S1 acumula 15 unidades de perda.

Proposta em avaliação: o limiar é sempre uma fração do ref.

- Na temporal, o padrão passa a ser o nível acumulado ($|x_S| / \text{ref}$).

- Na estática, compara-se a carga imediata ÷ ref.

Ajustes associados:

- ref > 0: hoje a validação só bloqueia ref = 0.
- Distinguir “ref em branco” de “ref = 1”.
- Nova tabela “variação em relação aos valores iniciais”: nível = ref + desvio, % de mudança sem e com resposta, e se o limiar foi cruzado.

Problema de fundo levantado pelo usuário:

- Na lógica original, o peso de uma aresta só servia para comparar arestas **do mesmo nível**.
- Ao ligar o limiar ao valor de referência em unidades do nó, os pesos passam a precisar ser comensuráveis **entre níveis**. Isso é, na prática, um problema de modelagem estrutural (path analysis ou SEM).
- O X2 propõe uma saída alinhada com essa lógica.

X2 — Modo estrutural: aresta = coeficiente de caminho padronizado β (adotado: D17–D23)

Ideia original (antes da decisão Q1, que deixou só o modo estrutural): dois modos de peso.

Modo	O que o peso significa	Como o motor usa
Relativo (atual)	Força comparada entre arestas do mesmo nível, em (0, 1]	<code>propagate()</code> com desconto $\lambda = c$ (D3)
Estrutural (opcional)	β : quantos desvios-padrão (DP) o destino muda quando a origem muda 1 DP	Efeito total pela álgebra de path analysis, sem desconto

X2.1 — Por que β , e não r^2

O r^2 mede quanto da variação do destino a origem explica, e não quanto o destino muda quando a origem muda, que é a pergunta de um cenário. Simulação em R de $y = b \cdot x + \text{ruído}$:

Caso	Slope b	r^2	β	Efeito de +1 unidade em x
A: efeito médio, pouco ruído	0,50	0,96	0,98	+0,50
B: mesmo efeito, muito ruído	0,51	0,06	0,24	+0,51
C: mesmo efeito, x variou pouco nos dados	0,51	0,04	0,20	+0,51
D: efeito minúsculo, sem ruído	0,05	0,96	0,98	+0,05

Além disso, o r^2 :

- não tem sinal;
- não se compõe ao longo dos caminhos: a path analysis multiplica coeficientes, não r^2 ;
- só soma o R^2 do nó quando as origens são independentes.

Slope bruto (b) e β são o mesmo modelo em unidades diferentes. $\beta = b \cdot \text{DP}_x / \text{DP}_y$, e mudar de um para o outro é mudar a escala dos nós. Conferido em R numa rede com ciclo: o raio espectral é idêntico (0,501 nos dois), e o efeito total também é idêntico depois de converter as unidades. A escolha entre b e β afeta só a entrada dos dados e a apresentação, nunca o resultado.

X2.2 — O que o usuário informa por aresta: dois campos

Campo	Uso
β , com sinal dado por <code>interaction_type</code>	Propagação
Incerteza: faixa de β	Reamostragem em p e na confiança da suficiência. Substitui <code>confidence</code> e é coerente com a D15

Atalhos do formulário. O formulário converte estes valores; eles não são guardados na aresta:

- $r^2 \rightarrow |\beta| = \sqrt{r^2}$ (vale para origens independentes; mostrar o aviso);
- r^2 e $n \rightarrow$ faixa de incerteza, com erro-padrão $\sqrt{((1 - r^2)/(n - 2))}$;
- slope bruto $\rightarrow \beta$, usando o ref e o DP dos dois nós, que pertencem à camada dos nós.

Sem dados: classe qualitativa (decidido em 29/09).

Classe	Faixa de β	Valor usado	Faixa de incerteza	r^2 equivalente
Fraca	< 0,3	0,15	0–0,3	< 0,09
Moderada (padrão)	0,3–0,6	0,45	0,3–0,6	0,09–0,36
Forte	0,6–1,0	0,80	0,6–1,0	0,36–1,0

- O sinal **nunca** tem padrão: é sempre informado (ver 0.7).
- Arestas com valor padrão ou classe são marcadas na tabela de arestas e no relatório, para deixar claro quanto do resultado depende de suposição.
- **Nunca usar $\beta = 1$ como padrão.** Isso seria uma transmissão perfeita: numa cadeia de 4 arestas o efeito chegaria inteiro à Resposta ($1^4 = 1$), e em rede com ciclo haveria risco de $p(B) \geq 1$.
- Numa cadeia de 4 arestas moderadas, o efeito total é $0,45^4 \approx 0,04$.

Ponte com o modo atual (álgebra, rede sem ciclo): o modo relativo equivale ao estrutural com $\beta = c \times \text{peso}$. Com peso 1 e $c = 0,5$, cada aresta vale $\beta = 0,5$, que fica dentro da classe moderada. Em rede com ciclo, $\lambda = c/\rho(W)$ desfaz essa equivalência exata.

X2.3 — Quantas “regressões” o usuário está fazendo

- **Um β por aresta; uma regressão por nó que recebe arestas.** Numa cadeia simples $D \rightarrow P \rightarrow S \rightarrow I \rightarrow R$, com uma origem por nó, são 4 β , o equivalente a 4 regressões simples.
- Num nó com várias origens ($P1 \rightarrow S$ e $P2 \rightarrow S$), os β saem de **uma regressão múltipla** de S sobre $P1$ e $P2$. Duas regressões simples contariam duas vezes o efeito comum, se $P1$ e $P2$ forem correlacionadas.
- Validação: a soma dos r^2 que chegam ≤ 1 por nó. O que falta para 1 é a variação vinda de fora da rede, mostrada como diagnóstico (“a rede explica 74% da variação do Estado”).
- Com o ciclo $I \rightarrow R \rightarrow P$, estimar cada equação por regressão comum fica enviesado; com dados reais, usar defasagem temporal ou SEM (ex.: lavaan). Para valores elicitados, isso não se aplica.

X2.4 — Efeito total, ciclos e coerência estática $\times$ temporal

- **Efeito total** = $(I - B)^{-1}p - p$: o produto dos β ao longo de cada caminho, somado entre caminhos. É a mesma forma de `propagate()`, com $\lambda = 1$. A atenuação de uma resposta distante (D3) vem do produto de $\beta < 1$, sem o desconto c .
- **Ciclos** exigem $p(B) < 1$, a verificar na validação.
- **Temporal:** com $x(t+1) = a \cdot x + B \cdot x + p$ e $a = 1 - \text{autorregulação}$, quando $a = 0$ a simulação converge exatamente para o efeito estático. Conferido em R: $I = 0,42$ a partir da janela 3. A leitura estática passa a ser o equilíbrio da temporal, e o conflito do X1 desaparece. O motor atual ($a = 1$, estoques que acumulam) continua no modo relativo.

Exemplo (conferido em R). Rede $P1 \rightarrow S$ ($\beta -0,7$), $P2 \rightarrow S$ ($-0,5$), $S \rightarrow I$ ($-0,6$) e $R \rightarrow P1$ ($-0,8$):

Leitura	Efeito de P1 em I	Efeito de R em I	Mitigação por R a 100%
Estrutural	+0,42 DP	-0,336 DP	80% (força para neutralizar: 125%)
App atual, peso = r^2 , $c = 0,5$	+0,044	-0,014	32% (força: 312%)

X2.5 — Camada dos nós, limiar e pressões acima de 100%

- Valor de referência e limiar ficam **só** na camada dos nós, para converter resultados para a unidade real. Eles nunca parametrizam arestas.
- Um nó com limiar ou com resultado em unidade real informa o valor inicial (ref) e a variação típica (DP, ou $CV = DP/ref$).
- **Não há teto de 0–1 ou 100%:**
 - a pressão é um deslocamento em DP, e o cenário aceita valores acima de 100% ou em unidade real;
 - o limiar é um nível real, então **deixa de ser validado em [0, 1]** no modo estrutural.
- **Exemplo de contaminação:** fundo de 10 mg/kg, DP 5, limite legal de 50 mg/kg (400% acima do fundo). O limiar fica em $z = (50 - 10)/5 = 8$ DP. Um lançamento de +200 mg/kg equivale a +40 DP.
- **Estoque de peixe:** ref 10, CV 20%, perda crítica de 40%, então $z = 0,4/0,2 = 2$ DP.
- Tabela de resultados: nível = ref + DP · z, com a variação em % do ref, sem e com resposta.
- **Cuidado:** β é linear. Muito além da faixa observada nos dados, a relação real costuma ser não linear (saturação, dose-resposta). O gatilho do limiar captura o liga/desliga, mas não a curvatura. Avisar quando o cenário passar muito da faixa dos dados.

X2.6 — Decisões

Tomadas em 29/09:

- β como parâmetro;
- incerteza como segundo campo;
- r^2 apenas como atalho e diagnóstico;
- classes fraca < 0,3, moderada 0,3–0,6 (padrão, 0,45), forte $\geq 0,6$.

Em aberto:

- adotar o modo estrutural nesta revisão ou numa futura;
- se o modo relativo continua disponível;
- se a temporal do modo estrutural usa a persistência $a = 1$ – autorregulação;
- como ficam a D3, a D7 e a D11 no modo estrutural;
- as decisões pendentes do X1 no modo relativo.

X3 — Acomodação do modo estrutural no roadmap (aplicada em 29/09)

O usuário decidiu adotar o modo estrutural (X2) nesta revisão. Esta seção registra a análise fase a fase que orientou a reescrita. **As decisões do X3.4 foram tomadas em 29/09** (Q1: só o estrutural; Q2: $a = 1 - sr$; Q3: confidence vira faixa de β ; Q4: exemplos só no estrutural) e aplicadas às fases. Com a Q2, parte do achado 3 do X3.1 vale só para nós com $sr = 1$ (ver D20); por isso os dois critérios de gatilho continuam, com o desvio acumulado como padrão (P1).

X3.1 — Três achados que simplificam a acomodação (conferidos em R)

1. **Um motor só.** Tudo pode operar sobre uma única matriz de efeitos B, com efeito estático $\Phi = (I - B)^{-1}p - p$.
 - No modo relativo, $B = \lambda W$ ($\lambda = c$, ou $c/p(W)$ com ciclo).
 - No modo estrutural, B é a matriz dos β , com $\lambda = 1$ e validação de $p(B) < 1$.
 - Suficiência, relevância, gatilho e caminhos passam a ser escritos uma vez, sobre B.

2. **O modo estrutural é Levins padronizado.** Com $\beta_{ij} = a_{ij} / |a_{ii}|$, o efeito estrutural é idêntico ao equilíbrio $-A^{-1}$ da especificação v1.0. No exemplo do PDF, os números são os mesmos: $D(I1) = 1,00$ e $D(I2) = 0,86$.
- A inversão registrada na B3 vinha de usar a_{ij} cru, sem dividir pela autorregulação.
 - **No modo estrutural, o conflito da D14 (Levins × propagação) desaparece.**
3. **O motor temporal já tem a forma certa.** Hoje o passo é $x(t+1) = (1 + g - sr) \cdot x + W \cdot x + p$. A persistência $a = 1 - sr + g$ já existe. O modo estrutural só precisa de um padrão diferente de autorregulação:
- com $a = 0$, sem memória, a simulação converge para a leitura estática;
 - além disso, “carga que chega ao Estado” e “nível do Estado” viram a mesma coisa, porque $x_S(t+1) = \sum \beta \cdot x_j(t) + p_S$;
 - **os dois critérios de gatilho da D11 coincidem quando $a = 0$** , e o conflito do X1 se resolve no modo estrutural.

X3.2 — Revisão fase a fase

Fase / item	Efeito do modo estrutural	Ação proposta
0 (0.1–0.8, bugs)	Independente	Mantém; fazer primeiro. As validações da 0.6/0.7 incluem os campos novos (β , faixa)
1.1 peso em (0, 1]	Compatível: $ \beta \leq 1$ com uma origem por nó	Mantém. O sinal continua em <code>interaction_type</code>
1.2 arquivos antigos	Compatível	Mantém. Arquivo antigo abre no modo relativo; converter para o estrutural é opcional
1.3 re-parametrizar exemplos	Muda: em vez de revisar peso por peso, cada aresta recebe uma classe (fraca/moderada/forte)	Fundir com a E7 (exemplos no modo estrutural)
A1–A2 modos e “até neutralizar”	Compatível. No modo estrutural com $a < 1$, o Impacto converge para o efeito estático do cenário, sem crescer sem limite. Pode ficar abaixo de zero se a resposta for mais que suficiente, mas de forma limitada	Mantém. Somar testes do modo estrutural à A6
A3 tolerância	Compatível	Mantém
A6 referências	Os valores atuais valem para o modo relativo	Mantém e adiciona: modo estrutural com $a = 0$ converge para o efeito estático
2.5 estabilizador λ	Só existe no modo relativo	Restringir ao modo relativo. No estrutural, bloquear se $p(B) \geq 1$
B1–B2 endpoint_class, v	Independentes	Mantém
B3 D pela propagação (D14)	No estrutural, $D = \text{Levins padronizado}$ (X3.1, achado 2)	Reescrever a D14 e o aviso: o aviso fica só no modo relativo. Refazer o teste de referência da B3 nos dois modos
B4 ρ (sinal estável)	Reamostragem passa a usar a faixa de β	Adaptar: faixa de β no estrutural, confidence no relativo
B5 eficácia e gap (D15)	Compatível	Mantém
B7 prioridade	Compatível	Mantém
C1–C3 gatilho (D7, D8, D11)	No estrutural: um critério só ($ x_S $ em DP comparado ao limiar z); estática e temporal coincidem com $a = 0$	Escrever o gatilho sobre B. D7/D11 continuam valendo para o relativo, com o X1 pendente só lá
C5 validação do limiar	No estrutural o limiar é um nível real, sem teto em 1	Adaptar a validação por modo
D1–D2 crescimento	Compatível; a tendência em DP é $\text{ref} \cdot g / DP = g / CV$	Mantém, com a fórmula corrigida (entrada constante)
2.3 / 2.7 caminhos	No estrutural, o produto dos β é o efeito exato do caminho	2.7: $\text{score do caminho} = \text{produto dos } \beta$. Mostrar o efeito por caminho
2.1, 2.2, 2.4, 2.6	Independentes	Mantém
3.10 validação dos modais	Campos novos	Incluir β , faixa, classe, atalhos r^2/n e $p(B)$
3.x demais	Independentes	Mantém
4 publicação	O tutorial ganha uma seção sobre o modo estrutural	Adicionar à A7 e à 1.3

X3.3 — Fase E (nova): modo estrutural

Aplicação (29/09): a Fase E não virou uma fase separada. E1–E3 e E7 foram para a nova Fase 1 (1.1–1.6), E5 foi para a C0, E4 foi para a Fase A e a 2.5, e E6 foi para a C0, a 1.1 e a 2.7. As referências abaixo a itens antigos (1.3 etc.) são da versão anterior do roadmap.

- **E1 — Dados e savepoint.**
 - `metadata$weight_mode` com os valores `relative` ou `structural`.
 - Arestas: `weight` passa a ser β no modo estrutural; novos `weight_low/weight_high` (faixa) e `strength_class` (fraca/moderada/forte, opcional).
 - Nós: `sd` ou `cv` opcional (camada de medida).
 - Savepoint antigo abre como `relative`.
- **E2 — Formulário de aresta.** Entrada por classe (padrão moderada, 0,45, faixa 0,3–0,6) ou por β direto. Atalhos: $r^2 \rightarrow |\beta|$ e $r^2 + n \rightarrow$ faixa. Arestas com valor padrão ficam marcadas.
- **E3 — Motor único sobre B.**
 - `effect_matrix(g, mode, c)` devolve B.
 - `propagate()` passa a receber B.
 - Validação: $p(B) < 1$ no estrutural, e soma de β^2 por nó ≤ 1 , com aviso.
 - Regressão: o modo relativo continua byte a byte igual.
- **E4 — Temporal no modo estrutural.** Persistência $a = 1 - sr + g$, com padrão a definir (X3.4, Q2). Sem o estabilizador λ .
- **E5 — Camada de medida e resultados em unidade real.**
 - O limiar vira nível real ($z = (\text{limiar} - \text{ref})/DP$), sem teto.
 - Pressão aceita valores acima de 100% ou em unidade real.
 - Nova tabela “variação em relação aos valores iniciais”: $\text{nível} = \text{ref} + DP \cdot z$.
 - Aviso de extrapolação linear.
- **E6 — Diagnósticos.** R^2 explicado por nó (“a rede explica X% da variação do Estado”) e efeito de cada caminho.
- **E7 — Exemplos.** Os três exemplos recebem classes no modo estrutural (substitui a 1.3). Tutorial com o exemplo de contaminação.

Ordem proposta: $0 \rightarrow 1.1/1.2 \rightarrow \text{E1–E3}$ (motor único) $\rightarrow A \rightarrow B \rightarrow C \rightarrow D \rightarrow \text{E4–E7} \rightarrow 2, 3, 4$. Com isso, cada fase de A a D é escrita uma vez, sobre B, e já serve aos dois modos.

X3.4 — Decisões necessárias antes de reescrever as fases

- **Q1 — O modo relativo continua?**
 - (a) Os dois modos, com o estrutural como padrão para projetos novos;
 - (b) só o estrutural, com o relativo apenas para abrir arquivos antigos. As classes qualitativas já cobrem o uso em oficinas sem dados, e o alcance c e a tabela “reach over c ” saem;
 - (c) os dois, com o relativo como padrão.
- **Q2 — Persistência padrão no modo estrutural.**
 - $a = 0$: sem memória; a temporal converge para a estática; nós respondem ao que chega a cada janela;
 - ou $a = 1 - sr$, com sr informado por nó: estoques que acumulam, como hoje.
- **Q3 — O que fazer com o confidence atual.** Mapear para a faixa de β (ex.: confiança 1 = faixa estreita) ou manter os dois campos separados.
- **Q4 — Exemplos.** Os três exemplos passam para o modo estrutural (classes) ou ficam nos dois modos para comparação.

X4 — Texto para o README e o tutorial: autorregulação (a aplicar na A7)

Rascunho em inglês, a língua do README. Entra no README (seção *Data format*, junto ao campo `self_regulation`) e no tutorial quando a Fase A estiver implementada: padrão 0,5, valor 1 aceito e motor sobre B. Os números foram conferidos em R (prototipos_revisao2/modo_estrutural.R, seção D20).

Self-regulation — how a factor recovers on its own. `self_regulation` (0–1, default 0.5) is the share of a factor’s deviation that fades by itself each window, independently of the network: 0 = nothing fades and everything that arrives accumulates (a stock with no replenishment, a persistent contaminant); 0.3 = 30% of the deviation fades

each window (a population rebuilding, a habitat regenerating); 1 = no memory, the factor only reflects what arrives in the current window.

Each window: $new\ value = (1 - self_regulation) \times previous\ value + what\ arrives\ through\ the\ edges + the\ push$.

Example — fishing effort (P) → fish stock (S) → catch decline (I), with $\beta = -0.7$ and -0.6 and a pressure of 1 SD:

Case	Window 2	4	8	12
Ongoing pressure, self-regulation 0 everywhere — fish stock	-0.70	-4.20	-19.6	-46.2
Ongoing pressure, stock recovers 30% per window — fish stock	-0.70	-1.53	-2.14	-2.29 (settles at $-0.7 \div 0.3 = -2.33$)
Ongoing pressure, self-regulation 1 everywhere — fish stock	-0.70	-0.70	-0.70	-0.70 (same as the static reading)
One-off pressure, stock recovers 30% per window — fish stock	-0.70	-0.34	-0.08	-0.02 (back to normal)

Under a constant load, a factor settles at $load \div self-regulation$: a contaminant with self-regulation 0.05 accumulates up to 20 times what arrives each window, which is how a small per-window pressure can still cross a threshold through accumulation.

Self-regulation	Half-life of recovery	Build-up at equilibrium ($1 \div self-regulation$)	Typical of
0.05	13.5 windows	20×	contaminant in sediment
0.10	6.6 windows	10×	reef recovery, long-lived species
0.30	1.9 windows	3×	short-cycle fish stock
0.50 (default)	1 window	2×	water quality that renews quickly
1	—	1×	monthly income, catch in the window

X5 — growth_rate × autorregulação (análise para a P3)

No motor atual, os dois são redundantes. O passo é $x(t+1) = (1 + g - sr) \cdot x + B \cdot x + p$: g e sr entram no mesmo lugar e só a diferença conta. Conferido com o motor real (simulate_temporal_pair()): g = 0,1 com sr = 0,3 dá exatamente o mesmo resultado, janela a janela, que g = 0 com sr = 0,2. Assim, um g positivo no desvio é só uma autorregulação negativa: um fator que amplifica o próprio desvio.

Conceitualmente, são coisas diferentes:

- **autorregulação:** como o **desvio** de um fator se desfaz (memória, recuperação), algo endógeno;
- **crecimento:** como o **nível de referência** de um fator se move por conta própria (crescimento populacional, tendência de consumo), algo exógeno, que existe com ou sem o cenário.

Proposta P3: separar os dois.

- g sai da diagonal. O passo do desvio fica $dev(t+1) = (1 - sr) \cdot dev + B \cdot x + p$.
- g vira só a tendência exógena da D10: $trend(t) = (ref/DP) \cdot ((1 + g)^t - 1)$, igual nas rodadas baseline e cenário.
- O valor do nó é $x = dev + trend$, e as arestas transmitem x. Assim, a influência de um fator em crescimento cresce na proporção $(1 + g)$, como a D9 quer, sem contar duas vezes, e a autorregulação não “come” a tendência.
- **O que muda na Fase D:**
 - a D2 volta a ter $trend(t)$ explícita, sem entrada somada ao termo $g \cdot x$;

- o teste D1 (“razão 1,10 com impulso em D”) passa a valer para a tendência, e não mais para o desvio de um impulso;
 - o teste D5 com D fora do cenário (0,05 / 0,105 / 0,1655) continua válido.
- **Validação:** $g > 0$ deixa de exigir $sr \geq g$, porque o crescimento não entra mais na estabilidade do desvio.